Universidad Nacional de Córdoba
Facultad de Ciencias Exactas, Físicas y Naturales

Sistemas de Computación
Trabajo Práctico N° 3

Docentes:
Ing. Miguel Ángel Solinas
Ing. Javier Alejandro Jorge

Autores:
Matías Lucas Morales Robles
José Ignacio Torres
Esteban Samuel Suarez

Índice

1. Introducción **2**

2. Arquitecturas de Firmware: UEFI y Coreboot **3**
2.1. UEFI (Unified Extensible Firmware Interface) . . . 3
2.2. Coreboot: La alternativa Open Source . . . 4
2.3. Experimentación práctica con el Master Boot Record (MBR) . . . 4
2.3.1. Creación de la imagen booteable . . . 4
2.3.2. Verificación del contenido binario . . . 5
2.3.3. Ejecución en QEMU . . . 5

3. Linker **6**
3.1. Linker Script utilizado . . . 6
3.2. Explicación del Linker Script . . . 6
3.2.1. Dirección de carga `0x7C00` . . . 6
3.2.2. Sección `.text` . . . 6
3.2.3. Firma de arranque `0xAA55` . . . 7

4. Código en Assembly **7**
4.1. Funcionamiento del programa . . . 7

5. Construcción y ejecución de la imagen booteable **8**
5.1. Ensamblado . . . 8
5.2. Enlace . . . 8
5.3. Ejecución en QEMU . . . 8

6. Comparación entre `objdump` y `hd` **10**
6.1. Análisis comparativo . . . 11
6.2. Relación entre offsets y direcciones de memoria . . . 12

7. Depuración con GDB **12**
7.1. Inicio de QEMU en modo depuración . . . 12
7.2. Conexión de GDB . . . 13
7.3. Configuración de breakpoints . . . 13

8. Desafío Final: Modo Protegido **15**

1. Introducción

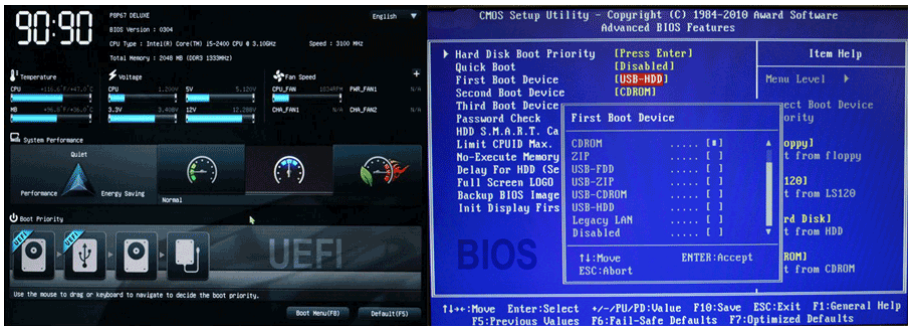
El estudio de la arquitectura x86 nos permite comprender cómo ha evolucionado el manejo de la memoria y la ejecución de procesos desde los primeros ordenadores personales hasta los sistemas modernos. En este contexto, es fundamental distinguir entre los dos estados operacionales primarios del procesador: el **Modo Real** y el **Modo Protegido**.

Originalmente, el Modo Real fue el estándar de los procesadores 8086, caracterizado por una gestión de memoria segmentada sin restricciones de acceso, lo que permitía una ejecución directa pero carente de seguridad. Con la llegada de procesadores más avanzados, se introdujo el Modo Protegido, una capacidad que desbloqueó la verdadera multitarea, el aislamiento de memoria y niveles de privilegio de hardware.

A continuación, se presenta una comparativa técnica de sus pilares fundamentales:

Característica	**Modo Real**	**Modo Protegido**
Ancho de palabra	16 bits	32 / 64 bits
Protección de Memoria	Inexistente	Implementada por Hardware
Capacidad de Multitarea	No nativa	Soporte completo y seguro
Acceso al Hardware	Directo y total	Controlado por el Kernel
Aislamiento de Procesos	Bajo / Nulo	Alto (Anillos de protección)

Cuadro 1: Comparativa entre estados de operación x86.



Actualmente, el Modo Real persiste principalmente como un estado transitorio durante el proceso de arranque (bootstrapping). Al encender el equipo, el procesador inicia en este modo para ejecutar el firmware inicial (BIOS/UEFI) y cargar el gestor de arranque, para luego realizar el salto definitivo hacia un entorno protegido donde el Sistema Operativo toma el control total.

2. Arquitecturas de Firmware: UEFI y Coreboot

2.1. UEFI (Unified Extensible Firmware Interface)

La Interfaz de Firmware Extensible Unificada (UEFI) se presenta como el estándar moderno que actúa como intermediario entre el hardware y el sistema operativo. Esta tecnología reemplaza al tradicional BIOS (*Basic Input/Output System*), el cual operaba fundamentalmente en texto, se ejecutaba en modo real e interactuaba mediante interrupciones de teclado. UEFI, por el contrario, permite inicializar los componentes del sistema de forma más eficiente y ofrece una interfaz operativa a la que comúnmente se accede mediante teclas como `F2`, `Del` o `Esc` durante el arranque, permitiendo configurar parámetros de hardware y secuencias de inicio en un entorno gráfico.

Característica	**BIOS Heredado**	**UEFI**
Origen	Década de 1980	Moderno (2005 en adelante)
Interfaz	Basada en texto	Gráfica y soporte para mouse
Límite de Disco	Hasta 2 TB (MBR)	Superior a 2 TB (GPT)
Velocidad de Arranque	Lenta	Rápida
Seguridad	Básica / Nula	*Secure Boot*
Arquitectura	16 bits (Modo Real)	32 / 64 bits (Modo Protegido)
Compatibilidad	Óptima con hardware antiguo	Orientada a sistemas modernos

Cuadro 2: Diferencias fundamentales entre BIOS y UEFI.

Servicios de UEFI: Una de las grandes ventajas de UEFI es que expone funciones (Runtime Services) que el sistema operativo puede invocar directamente una vez iniciado:

- **GetTime() / SetTime():** Permiten leer y modificar, respectivamente, la fecha y hora del reloj de tiempo real (RTC) del sistema, siempre que las políticas de seguridad del firmware lo permitan.
- **GetVariable() / SetVariable():** Encargadas de leer, crear o actualizar variables almacenadas en la memoria no volátil (NVRAM). Son cruciales para mantener configuraciones persistentes, como las entradas del gestor de arranque.
- **GetNextVariableName():** Utilizada para iterar y enumerar todas las variables disponibles en la NVRAM.
- **ResetSystem():** Proporciona un mecanismo estandarizado para reiniciar o apagar el equipo de forma controlada desde el propio firmware.

Vulnerabilidades en UEFI: A pesar de sus mejoras de seguridad, la complejidad de UEFI ha introducido nuevos vectores de ataque. Un ejemplo notable es la vulnerabilidad conocida como **Boothole** (descubierta en 2020). Este fallo residía en el cargador GRUB2 y comprometía la cadena de confianza de *Secure Boot*, permitiendo a un atacante ejecutar código arbitrario antes de la carga del sistema operativo. Otros fallos comunes suelen derivar de una gestión insegura de las variables en la NVRAM o de controladores UEFI mal implementados.

Gestión de Hardware en Intel: CSME y MEBx En ecosistemas Intel, la administración de la plataforma se apoya en el **CSME** (*Converged Security and Management Engine*). Este componente es un microprocesador embebido dentro del chipset que opera de manera totalmente independiente de la CPU principal, encargándose de la seguridad y la gestión remota. Para su configuración, se utiliza el **MEBx** (*Intel Management Engine BIOS Extension*), una interfaz que permite establecer parámetros de red, credenciales y opciones de administración fuera de banda.

2.2. Coreboot: La alternativa Open Source

En contraposición a las soluciones propietarias, **Coreboot** nace como un proyecto de firmware de código abierto diseñado para reemplazar al BIOS o UEFI tradicional. Su premisa fundamental es inicializar el silicio y el hardware de la manera más expedita posible para delegar el control a un *payload* (un cargador de arranque o incluso un kernel del sistema operativo).

Empresas e iniciativas enfocadas en la privacidad y la tecnología abierta, como las *Chromebooks*, *System76*, *Purism Librem*, *PC Engines APU* y *Raptor Computing Systems*, utilizan Coreboot por defecto.

Ventaja	Descripción Técnica
Arranque Optimizado	Reduce drásticamente el tiempo de inicio al realizar inicializaciones estrictamente necesarias.
Auditable y Abierto	Al ser *open source*, la comunidad puede inspeccionar, depurar y mejorar el código fuente.
Control de Hardware	Otorga al usuario la capacidad de personalizar qué componentes se activan durante el arranque.
Mayor Seguridad	Mitiga el riesgo de puertas traseras (*backdoors*) y vulnerabilidades ocultas presentes en el firmware cerrado.
Flexibilidad de Payload	Compatible con múltiples *payloads* como SeaBIOS, Tianocore (para soporte UEFI) o LinuxBoot.
Huella Reducida	El binario compilado ocupa una fracción del espacio requerido por un firmware comercial.

Cuadro 3: Principales beneficios del uso de Coreboot.

2.3. Experimentación práctica con el Master Boot Record (MBR)

A nivel de bajo nivel, es posible construir manualmente una imagen de arranque respetando la estructura clásica del *Master Boot Record* (MBR). Para ello, se generó un archivo binario que contiene una instrucción ejecutable, el relleno necesario hasta completar un sector de 512 bytes y la firma de arranque requerida por la BIOS.

2.3.1. Creación de la imagen booteable

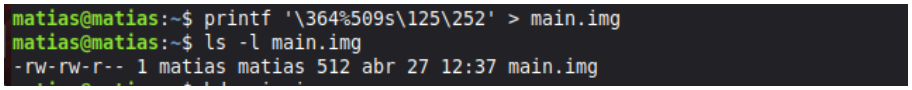
La imagen se generó mediante el siguiente comando:

```
printf '\364%509s\125\252' > main.img
```

El comando construye un sector de arranque válido de exactamente 512 bytes. Su funcionamiento es el siguiente:

- `\364`: representa el valor octal correspondiente al opcode `0xF4`, que implementa la instrucción `HLT` (*Halt*). Esta instrucción detiene al procesador inmediatamente después de iniciar la ejecución.
- `%509s`: inserta 509 bytes de relleno, completando el sector hasta la posición 510.
- `\125\252`: corresponden a los valores octales `0x55` y `0xAA`. Estos dos bytes forman la firma de arranque obligatoria (`0xAA55`) ubicada en los últimos dos bytes del sector.

Sin esta firma, la BIOS no reconocería la imagen como un dispositivo de arranque válido.



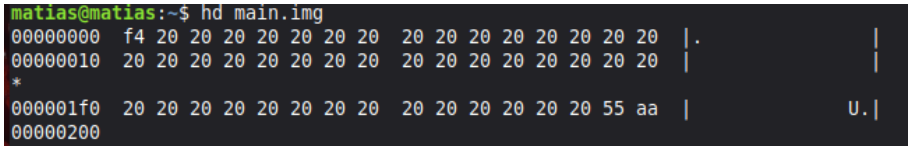
2.3.2. Verificación del contenido binario

Para verificar que los bytes fueron escritos correctamente y que la firma de arranque se encuentra en la posición adecuada, se utilizó el comando:

```
hd main.img
```

Este volcado hexadecimal permite comprobar que:

- El primer byte es `F4`, correspondiente a la instrucción `HLT`.
- Los bytes intermedios contienen el relleno necesario.
- Los dos últimos bytes son `55 AA`, que constituyen la firma de arranque.

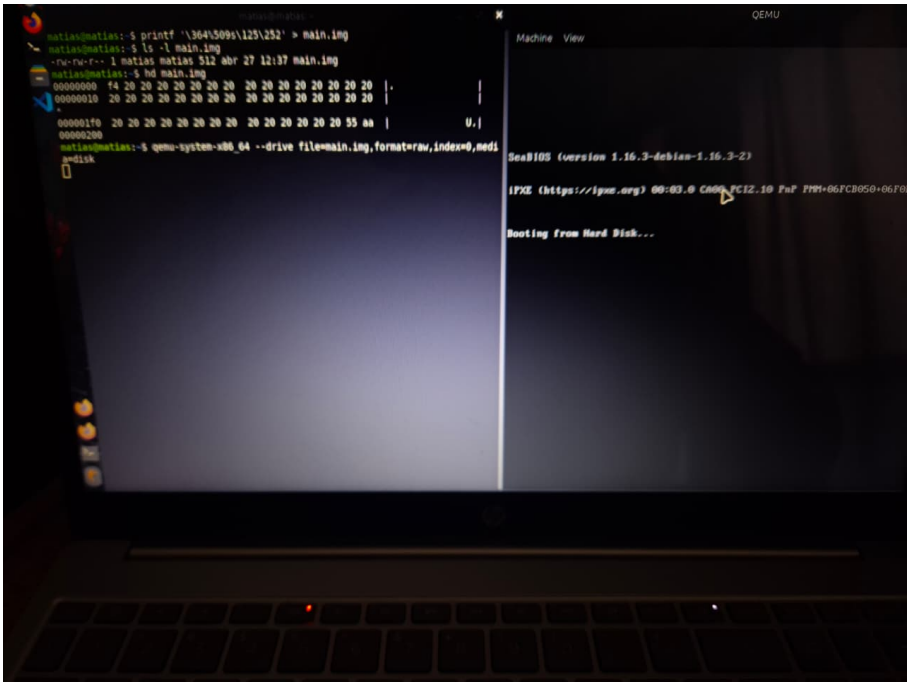


2.3.3. Ejecución en QEMU

Finalmente, la imagen fue ejecutada mediante el emulador QEMU utilizando el siguiente comando:

```
qemu-system-x86_64 --drive file=main.img,format=raw,index=0,media=disk
```

Al iniciarse, la BIOS detecta la firma de arranque, carga el sector en la dirección `0x7C00` y transfiere allí la ejecución. Como la primera instrucción es `HLT`, el procesador se detiene inmediatamente.



3. Linker

El *linker* (o enlazador) es una herramienta encargada de tomar uno o más archivos objeto generados durante la compilación o el ensamblado, y combinarlos en un único archivo ejecutable o imagen binaria. Entre sus funciones principales se encuentran la resolución de referencias a símbolos (funciones, variables o etiquetas) entre distintos módulos y la asignación de direcciones de memoria a las diferentes secciones del programa.

En el contexto de un sector de arranque, el linker cumple un rol fundamental, ya que debe generar una imagen binaria plana, sin cabeceras ni metadatos adicionales, que pueda ser cargada directamente por el BIOS.

La dirección especificada en el *linker script* indica la posición de memoria en la que el programa será cargado y ejecutado. Esto permite que el código y los datos queden ubicados correctamente en memoria, de acuerdo con las convenciones de arranque de la arquitectura x86.

3.1. Linker Script utilizado

El siguiente *linker script* define la organización de la imagen binaria generada y la ubicación exacta de cada sección en memoria:

```
SECTIONS
{
    . = 0x7C00;
    .text :
    {
        __start = .;
        *(.text)
        . = 0x1FE;
        SHORT(0xAA55)
    }
}
```

Listing 1: Archivo `link.ld`

3.2. Explicación del Linker Script

El script de enlace define cómo debe construirse la imagen booteable y en qué posiciones de memoria se ubicará cada parte del programa. Esto resulta esencial en el desarrollo de un sector de arranque, ya que el BIOS espera encontrar tanto el código como la firma de arranque en ubicaciones específicas.

3.2.1. Dirección de carga `0x7C00`

La instrucción

```
. = 0x7C00;
```

establece que el código del programa será ubicado a partir de la dirección física `0x7C00`. Esta es la dirección estándar en la que el BIOS carga en memoria el primer sector de arranque (512 bytes) de un dispositivo booteable.

Una vez cargado, el procesador transfiere la ejecución a esa dirección, por lo que el código debe estar preparado para ejecutarse correctamente desde allí.

3.2.2. Sección `.text`

La sección

```
.text :
{
    __start = .;
    *(.text)
    ...
}
```

indica que todas las instrucciones del programa ensamblado se ubicarán dentro de la sección `.text`. La directiva

```
__start = .;
```

define una etiqueta simbólica en el inicio del código, lo que permite referenciar posteriormente la dirección de comienzo del programa si fuera necesario.

3.2.3. Firma de arranque `0xAA55`

Las instrucciones

```
. = 0x1FE;
SHORT(0xAA55)
```

desplazan la posición actual hasta el byte 510 del sector de arranque e insertan allí el valor `0xAA55`. Debido al formato little-endian de la arquitectura x86, este valor se almacena en memoria como los bytes `55 AA`.

Esta secuencia constituye la *boot signature* o firma de arranque. El BIOS verifica obligatoriamente la presencia de esta firma en los dos últimos bytes del sector. Si no se encuentra en esa posición exacta, el dispositivo no será reconocido como booteable y el proceso de arranque fallará.

4. Código en Assembly

El programa fue desarrollado en lenguaje ensamblador para la arquitectura x86 en modo real de 16 bits. Su objetivo es imprimir en pantalla el mensaje `hello world` utilizando los servicios de video provistos por el BIOS.

Para ello, se emplea la interrupción `INT 0x10`, específicamente la función `0x0E`, conocida como *teletype output*. Esta función permite mostrar caracteres en pantalla de manera secuencial, avanzando automáticamente la posición del cursor.

El código fuente utilizado se muestra a continuación:

```
.code16
    mov $msg, %si
    mov $0x0e, %ah
loop:
    lodsb
    or %al, %al
    jz halt
    int $0x10
    jmp loop
halt:
    hlt
msg:
    .asciz "hello world"
```

Listing 2: Archivo `main.s`

4.1. Funcionamiento del programa

Al comenzar la ejecución, se carga en el registro `SI` la dirección de memoria correspondiente al inicio de la cadena `msg`. Este registro actuará como puntero al texto que se desea imprimir.

Luego, se almacena en el registro `AH` el valor `0x0E`, que selecciona la función de salida de caracteres del BIOS mediante la interrupción `INT 0x10`.

El programa ingresa entonces en un bucle que se ejecuta carácter por carácter:

- La instrucción `lodsb` carga en el registro `AL` el byte apuntado por `SI` y luego incrementa automáticamente dicho registro.
- La instrucción `or %al, %al` se utiliza para verificar si el carácter leído es nulo (`0x00`), lo que indica el final de la cadena.

- Si se alcanza el terminador nulo, la instrucción `jz halt` transfiere el control a la rutina de finalización.
- En caso contrario, la interrupción `INT 0x10` imprime el carácter almacenado en `AL`.
- Finalmente, la instrucción `jmp loop` repite el proceso para el siguiente carácter.

Una vez impreso el mensaje completo, la instrucción `hlt` detiene la ejecución del procesador.

5. Construcción y ejecución de la imagen booteable

Para generar la imagen de arranque y ejecutarla en una máquina virtual, se siguieron tres etapas fundamentales: ensamblado, enlace y ejecución. Cada una de ellas cumple un rol específico dentro del proceso de construcción del sistema booteable.

5.1. Ensamblado

El primer paso consiste en traducir el código fuente en ensamblador a un archivo objeto. Para ello se utiliza el ensamblador de GNU mediante el siguiente comando:

```
as -g -o main.o main.s
```

En este comando:

- `-g` agrega información de depuración, útil para el análisis posterior con herramientas como GDB.
- `-o main.o` especifica el nombre del archivo objeto de salida.
- `main.s` es el archivo fuente escrito en lenguaje ensamblador.

El resultado es el archivo `main.o`, que contiene el código ensamblado en formato objeto relocatable. En esta etapa, el programa aún no puede ejecutarse directamente.

5.2. Enlace

Una vez generado el archivo objeto, se procede a enlazarlo para obtener una imagen binaria plana apta para el arranque. El comando utilizado es:

```
ld --oformat binary -o main.img -T link.ld main.o
```

En este comando:

- `-oformat binary` indica que la salida debe ser un binario plano, sin cabeceras ni información adicional.
- `-o main.img` especifica el nombre del archivo de salida.
- `-T link.ld` indica que se utilizará el *linker script* personalizado.
- `main.o` es el archivo objeto generado en la etapa anterior.

Como resultado, se obtiene el archivo `main.img`, una imagen booteable de 512 bytes que contiene tanto el código del programa como la firma de arranque `0xAA55` en la posición requerida por el BIOS.

5.3. Ejecución en QEMU

Para verificar el funcionamiento de la imagen generada, se utilizó QEMU, un emulador de sistemas capaz de ejecutar software a bajo nivel. El comando empleado fue:

```
qemu-system-i386 -drive format=raw,file=main.img
```

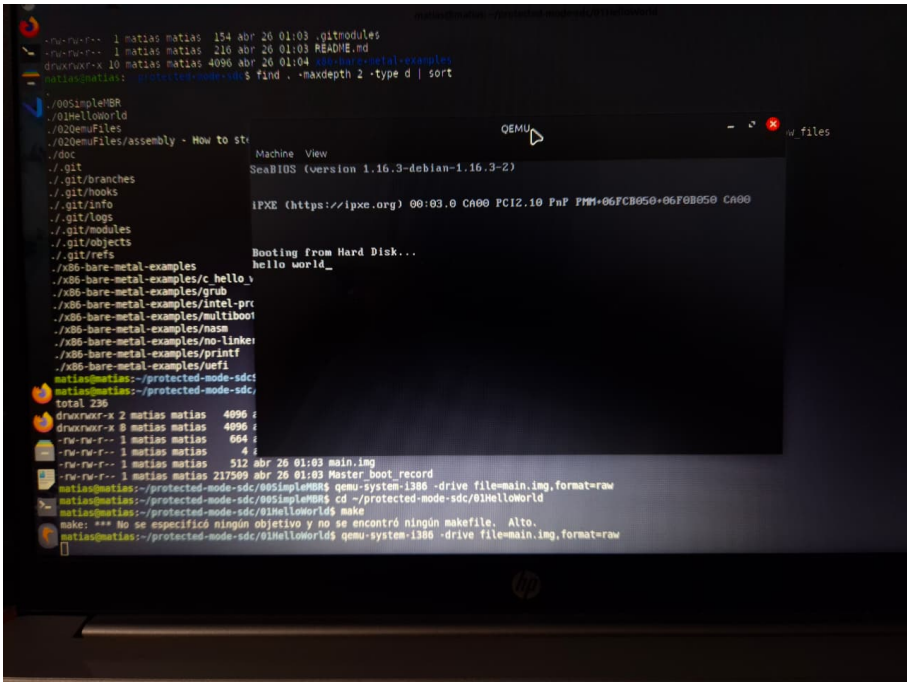
En este comando:

- `qemu-system-i386` emula una computadora basada en la arquitectura x86 de 32 bits.

- `-drive format=raw,file=main.img` indica que la imagen a utilizar es un archivo binario plano en formato RAW.

Durante el arranque, el BIOS virtual carga el contenido de `main.img` en la dirección de memoria `0x7C00`, verifica la firma de arranque y transfiere el control al código del bootloader.

Como resultado, el programa se ejecuta en modo real de 16 bits e imprime en pantalla el mensaje `hello world` utilizando la interrupción de video del BIOS.



6. Comparación entre `objdump` y `hd`

Para analizar la estructura interna de la imagen booteable generada, se emplearon dos herramientas complementarias: `hd` y `objdump`.

El comando `hd` permite visualizar el contenido del archivo en formato hexadecimal, mostrando los bytes exactos almacenados en la imagen. Por su parte, `objdump` interpreta esos mismos bytes como instrucciones de máquina y presenta su correspondiente desensamblado para la arquitectura x86 en modo real de 16 bits.

```
matias@matias:~/protected-mode-sdc/01HelloWorld$ hd main.img
00000000  be 0f 7c b4 0e ac 08 c0  74 04 cd 10 eb f7 f4 68  |..|.....t......h|
00000010  65 6c 6c 6f 20 77 6f 72  6c 64 00 66 2e 0f 1f 84  |ello world.f....|
00000020  00 00 00 00 00 66 2e 0f  1f 84 00 00 00 00 00 66  |.....f.........f|
00000030  2e 0f 1f 84 00 00 00 00  00 66 2e 0f 1f 84 00 00  |.........f......|
00000040  00 00 00 66 2e 0f 1f 84  00 00 00 00 00 66 2e 0f  |...f.........f..|
00000050  1f 84 00 00 00 00 00 66  2e 0f 1f 84 00 00 00 00  |.......f........|
00000060  00 66 2e 0f 1f 84 00 00  00 00 00 66 2e 0f 1f 84  |.f.........f....|
00000070  00 00 00 00 00 66 2e 0f  1f 84 00 00 00 00 00 66  |.....f.........f|
00000080  2e 0f 1f 84 00 00 00 00  00 66 2e 0f 1f 84 00 00  |.........f......|
00000090  00 00 00 66 2e 0f 1f 84  00 00 00 00 00 66 2e 0f  |...f.........f..|
000000a0  1f 84 00 00 00 00 00 66  2e 0f 1f 84 00 00 00 00  |.......f........|
000000b0  00 66 2e 0f 1f 84 00 00  00 00 00 66 2e 0f 1f 84  |.f.........f....|
000000c0  00 00 00 00 00 66 2e 0f  1f 84 00 00 00 00 00 66  |.....f.........f|
000000d0  2e 0f 1f 84 00 00 00 00  00 66 2e 0f 1f 84 00 00  |.........f......|
000000e0  00 00 00 66 2e 0f 1f 84  00 00 00 00 00 66 2e 0f  |...f.........f..|
000000f0  1f 84 00 00 00 00 00 66  2e 0f 1f 84 00 00 00 00  |.......f........|
00000100  00 66 2e 0f 1f 84 00 00  00 00 00 66 2e 0f 1f 84  |.f.........f....|
00000110  00 00 00 00 00 66 2e 0f  1f 84 00 00 00 00 00 66  |.....f.........f|
00000120  2e 0f 1f 84 00 00 00 00  00 66 2e 0f 1f 84 00 00  |.........f......|
00000130  00 00 00 66 2e 0f 1f 84  00 00 00 00 00 66 2e 0f  |...f.........f..|
00000140  1f 84 00 00 00 00 00 66  2e 0f 1f 84 00 00 00 00  |.......f........|
00000150  00 66 2e 0f 1f 84 00 00  00 00 00 66 2e 0f 1f 84  |.f.........f....|
00000160  00 00 00 00 00 66 2e 0f  1f 84 00 00 00 00 00 66  |.....f.........f|
00000170  2e 0f 1f 84 00 00 00 00  00 66 2e 0f 1f 84 00 00  |.........f......|
00000180  00 00 00 66 2e 0f 1f 84  00 00 00 00 00 66 2e 0f  |...f.........f..|
00000190  1f 84 00 00 00 00 00 66  2e 0f 1f 84 00 00 00 00  |.......f........|
000001a0  00 66 2e 0f 1f 84 00 00  00 00 00 66 2e 0f 1f 84  |.f.........f....|
000001b0  00 00 00 00 00 66 2e 0f  1f 84 00 00 00 00 00 66  |.....f.........f|
000001c0  2e 0f 1f 84 00 00 00 00  00 66 2e 0f 1f 84 00 00  |.........f......|
000001d0  00 00 00 66 2e 0f 1f 84  00 00 00 00 00 66 2e 0f  |...f.........f..|
000001e0  1f 84 00 00 00 00 00 66  2e 0f 1f 84 00 00 00 00  |.......f........|
000001f0  00 66 2e 0f 1f 84 00 00  00 00 00 0f 1f 00 55 aa  |.f............U.|
00000200
matias@matias:~/protected-mode-sdc/01HelloWorld$
```

Figura 1: Hexdump de la imagen booteable `main.img`.

```
matias@matias:~/protected-mode-sdc/01HelloWorld$ objdump -D -b binary -mi8086 -Maddr16,data16 main.img

main.img:     formato del fichero binary


Desensamblado de la sección .data:

00000000 <.data>:
   0:   be 0f 7c                mov    $0x7c0f,%si
   3:   b4 0e                   mov    $0xe,%ah
   5:   ac                      lods   %ds:(%si),%al
   6:   08 c0                   or     %al,%al
   8:   74 04                   je     0xe
   a:   cd 10                   int    $0x10
   c:   eb f7                   jmp    0x5
   e:   f4                      hlt
   f:   68 65 6c                push   $0x6c65
  12:   6c                      insb   (%dx),%es:(%di)
  13:   6f                      outsw  %ds:(%si),(%dx)
  14:   20 77 6f                and    %dh,0x6f(%bx)
  17:   72 6c                   jb     0x85
  19:   64 00 66 2e             add    %ah,%fs:0x2e(%bp)
  1d:   0f 1f 84 00 00          nopw   0x0(%si)
  22:   00 00                   add    %al,(%bx,%si)
  24:   00 66 2e                add    %ah,0x2e(%bp)
  27:   0f 1f 84 00 00          nopw   0x0(%si)
  2c:   00 00                   add    %al,(%bx,%si)
  2e:   00 66 2e                add    %ah,0x2e(%bp)
  31:   0f 1f 84 00 00          nopw   0x0(%si)
  36:   00 00                   add    %al,(%bx,%si)
  38:   00 66 2e                add    %ah,0x2e(%bp)
  3b:   0f 1f 84 00 00          nopw   0x0(%si)
  40:   00 00                   add    %al,(%bx,%si)
  42:   00 66 2e                add    %ah,0x2e(%bp)
  45:   0f 1f 84 00 00          nopw   0x0(%si)
  4a:   00 00                   add    %al,(%bx,%si)
  4c:   00 66 2e                add    %ah,0x2e(%bp)
  4f:   0f 1f 84 00 00          nopw   0x0(%si)
  54:   00 00                   add    %al,(%bx,%si)
  56:   00 66 2e                add    %ah,0x2e(%bp)
  59:   0f 1f 84 00 00          nopw   0x0(%si)
```

Figura 2: Desensamblado de la imagen `main.img` utilizando `objdump`.

6.1. Análisis comparativo

Las Figuras 1 y 2 muestran, respectivamente, el contenido hexadecimal de la imagen booteable y su desensamblado para la arquitectura Intel 8086.

En la Figura 1 se observan los bytes crudos almacenados en el archivo `main.img`. Por su parte, la Figura 2 interpreta esos mismos bytes como instrucciones ejecutables en modo real de 16 bits.

La correspondencia entre ambas representaciones se resume en la Tabla 2.

Offset	Bytes	Instrucción
0x00	be 0f 7c	mov $0x7c0f,%si
0x03	b4 0e	mov $0x0e,%ah
0x05	ac	lodsb
0x06	08 c0	or %al,%al
0x08	74 04	jz 0x0e
0x0A	cd 10	int $0x10
0x0C	eb f7	jmp 0x05

Cuadro 4: Correspondencia entre el hexdump y el desensamblado

Cada instrucción mostrada por `objdump` se corresponde exactamente con la secuencia de bytes observada en el hexdump. Esto confirma que la imagen binaria fue generada correctamente y que el código ensamblado fue traducido sin alteraciones.

A partir del offset `0x0E` comienzan los datos correspondientes a la cadena `hello world`, almacenada en formato ASCII y terminada con un byte nulo (`0x00`), tal como requiere la directiva `.asciz`.

Asimismo, en el offset `0x1FE` se encuentra la secuencia `55 aa`, que corresponde a la firma de arranque exigida por el BIOS. Esta firma valida a `main.img` como un sector de arranque legítimo y habilita su ejecución durante el proceso de booteo.

6.2. Relación entre offsets y direcciones de memoria

El comando `hd` muestra el contenido de `main.img` como una secuencia lineal de bytes almacenados en el archivo, comenzando en el offset `0x00`. Este offset representa la posición relativa de cada byte dentro de la imagen en disco.

Sin embargo, durante el proceso de arranque, el BIOS carga el contenido completo del sector en la memoria RAM a partir de la dirección física `0x7C00`. Por lo tanto, existe una correspondencia directa entre los offsets del archivo y las direcciones de memoria una vez cargado.

En particular:

- El byte ubicado en el offset `0x00` del archivo se carga en la dirección `0x7C00`.
- El byte en el offset `0x01` se ubica en `0x7C01`.
- En general, un byte en el offset `x` del archivo se cargará en la dirección `0x7C00 + x`.

Por ejemplo, la firma de arranque ubicada en el offset `0x1FE` del archivo se encontrará en la dirección física `0x7DFE` una vez cargada en memoria.

Esta relación explica por qué `objdump` muestra instrucciones referenciando direcciones cercanas a `0x7C00`, como en el caso de `mov $0x7c0f,%si`, donde se carga la dirección absoluta de la cadena `msg` en memoria.

7. Depuración con GDB

Para analizar la ejecución del bootloader paso a paso, se utilizó el depurador GNU Debugger (GDB) en conjunto con QEMU. Esta combinación permite inspeccionar el estado del procesador, la memoria y la ejecución de las instrucciones en tiempo real.

QEMU ofrece soporte nativo para depuración remota mediante el protocolo GDB, lo que posibilita conectar el depurador a la máquina virtual mientras esta ejecuta el código del sector de arranque.

7.1. Inicio de QEMU en modo depuración

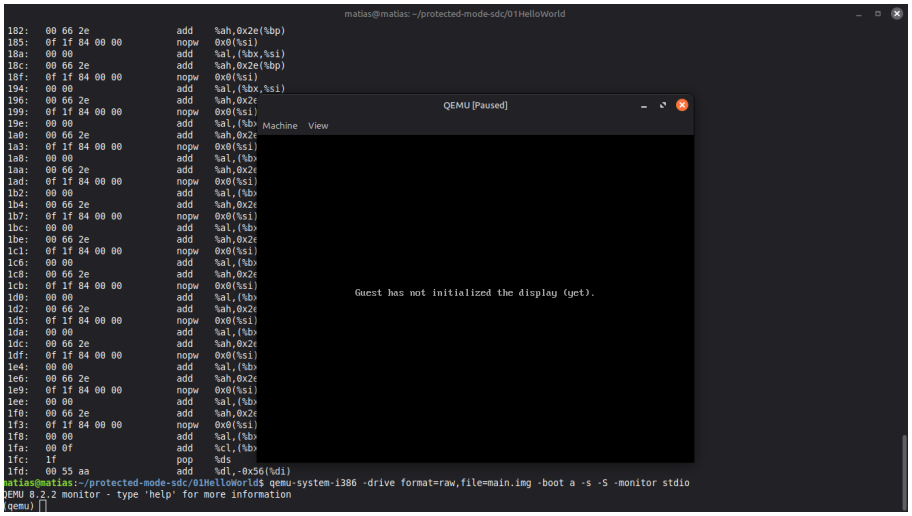
Para permitir la conexión de GDB, QEMU debe iniciarse con opciones especiales. El comando utilizado fue:

```
qemu-system-i386 -drive format=raw,file=main.img -boot a -s -S -monitor stdio
```

En este comando:

- `-drive format=raw,file=main.img` especifica la imagen booteable a ejecutar.
- `-boot a` indica que el arranque debe realizarse desde la unidad de disquete virtual.
- `-s` inicia un servidor GDB en el puerto `1234`.
- `-S` mantiene la CPU detenida al inicio, esperando la conexión del dcepurador.
- `-monitor stdio` habilita la consola interactiva de QEMU en la terminal.

En estas condiciones, QEMU queda en pausa inmediatamente después de iniciarse, aguardando que GDB se conecte antes de comenzar la ejecución del bootloader.



7.2. Conexión de GDB

Una vez iniciado QEMU en modo depuración, se abre una nueva terminal y se ejecuta GDB:

```
gdb
```

Luego, para conectar el depurador con la máquina virtual, se utiliza el comando:

```
(gdb) target remote localhost:1234
```

Este comando establece una conexión remota con el servidor GDB que QEMU expone en el puerto `1234`.

A continuación, es necesario configurar GDB para interpretar correctamente las instrucciones del procesador en modo real de 16 bits:

```
(gdb) set architecture i8086
```

De esta manera, el depurador desensamblará y mostrará las instrucciones utilizando la arquitectura Intel 8086, que es la empleada durante las primeras etapas del proceso de arranque.

7.3. Configuración de breakpoints

Para analizar el comportamiento del bootloader, se establecieron dos puntos de interrupción en direcciones clave del programa:

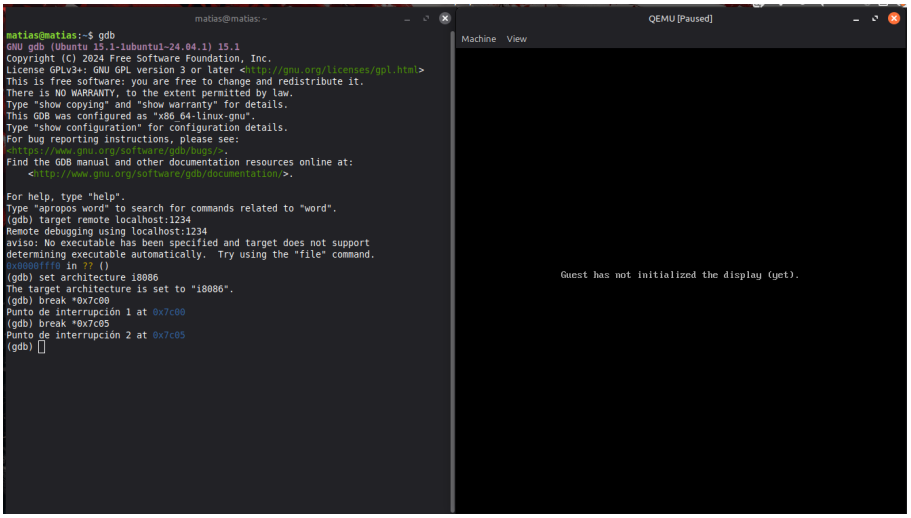
- `0x7C00`: inicio del sector de arranque, donde comienza la ejecución del bootloader.
- `0x7C05`: comienzo del bucle encargado de imprimir el mensaje en pantalla.

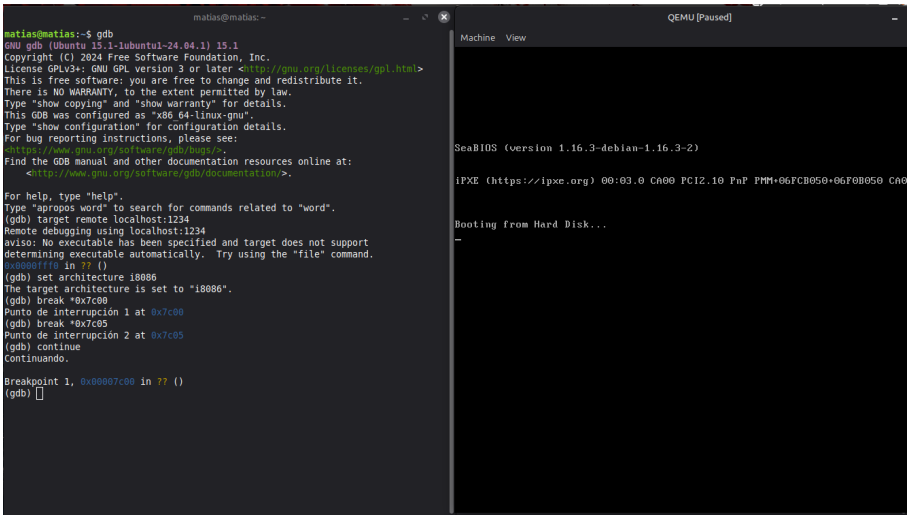
Los comandos utilizados fueron:

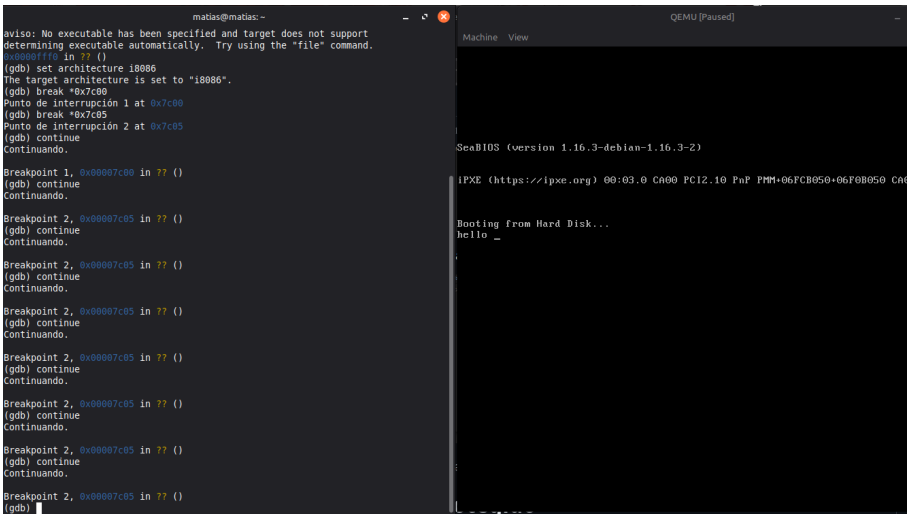
```
(gdb) break *0x7c00
(gdb) break *0x7c05
(gdb) continue
```

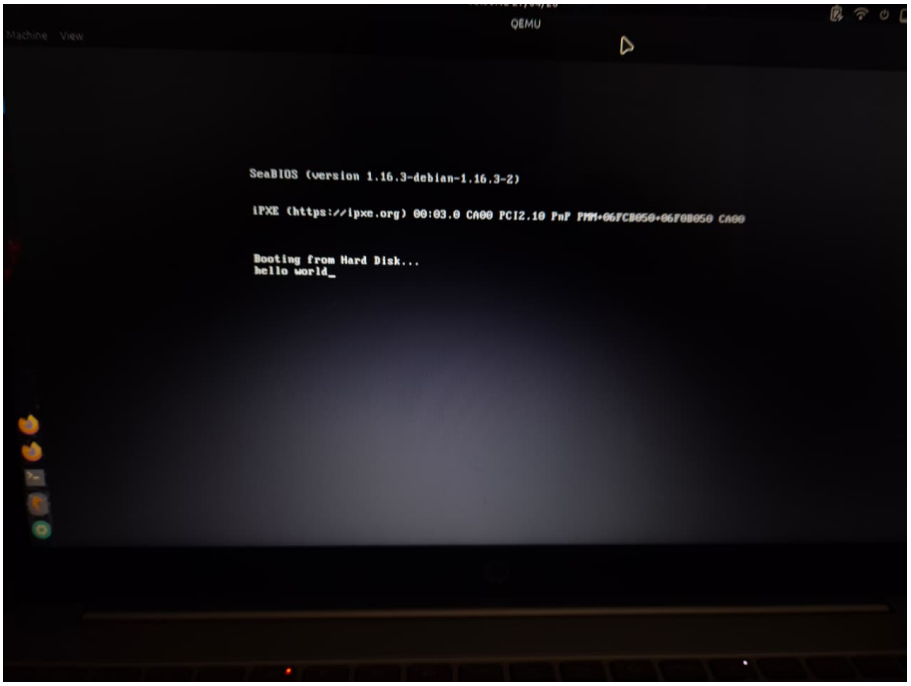
El primer breakpoint permite verificar que el BIOS ha cargado correctamente el sector de arranque en la dirección esperada y ha transferido el control al programa.

El segundo breakpoint detiene la ejecución justo antes de que se inicie la lectura e impresión de la cadena `hello world`, permitiendo observar en detalle el funcionamiento del bucle principal.


matias@matias: ~
matias@matias:~$ gdb
GNU gdb (Ubuntu 15.1-1ubuntu1~24.04.1) 15.1
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word".
(gdb) target remote localhost:1234
Remote debugging using localhost:1234
aviso: No executable has been specified and target does not support
determining executable automatically. Try using the "file" command.
0x0000fff0 in ?? ()
(gdb) set architecture i8086
The target architecture is set to "i8086".
(gdb) break *0x7c00
Punto de interrupción 1 at 0x7c00
(gdb) break *0x7c05
Punto de interrupción 2 at 0x7c05
(gdb)
QEMU [Paused]
Machine View
Guest has not initialized the display (yet).


matias@matias: ~
matias@matias:~$ gdb
GNU gdb (Ubuntu 15.1-1ubuntu1~24.04.1) 15.1
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word".
(gdb) target remote localhost:1234
Remote debugging using localhost:1234
aviso: No executable has been specified and target does not support
determining executable automatically. Try using the "file" command.
0x0000fff0 in ?? ()
(gdb) set architecture i8086
The target architecture is set to "i8086".
(gdb) break *0x7c00
Punto de interrupción 1 at 0x7c00
(gdb) break *0x7c05
Punto de interrupción 2 at 0x7c05
(gdb) continue
Continuando.
Breakpoint 1, 0x00007c00 in ?? ()
(gdb)
QEMU [Paused]
Machine View
SeaBIOS (version 1.16.3-debian-1.16.3-2)
iPXE (https://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+06FCB050+06F0B050 CA0
Booting from Hard Disk...


matias@matias: ~
aviso: No executable has been specified and target does not support
determining executable automatically. Try using the "file" command.
0x0000fff0 in ?? ()
(gdb) set architecture i8086
The target architecture is set to "i8086".
(gdb) break *0x7c00
Punto de interrupción 1 at 0x7c00
(gdb) break *0x7c05
Punto de interrupción 2 at 0x7c05
(gdb) continue
Continuando.
Breakpoint 1, 0x00007c00 in ?? ()
(gdb) continue
Continuando.
Breakpoint 2, 0x00007c05 in ?? ()
(gdb) continue
Continuando.
Breakpoint 2, 0x00007c05 in ?? ()
(gdb) continue
Continuando.
Breakpoint 2, 0x00007c05 in ?? ()
(gdb) continue
Continuando.
Breakpoint 2, 0x00007c05 in ?? ()
(gdb) continue
Continuando.
Breakpoint 2, 0x00007c05 in ?? ()
(gdb) continue
Continuando.
Breakpoint 2, 0x00007c05 in ?? ()
(gdb) continue
Continuando.
Breakpoint 2, 0x00007c05 in ?? ()
(gdb)
QEMU [Paused]
Machine View
SeaBIOS (version 1.16.3-debian-1.16.3-2)
iPXE (https://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+06FCB050+06F0B050 CA0
Booting from Hard Disk...
hello



8. Desafío Final: Modo Protegido

La primera parte de este desafío consiste en desarrollar un programa en lenguaje ensamblador capaz de efectuar la transición al modo protegido prescindiendo de las macros facilitadas en el código base. Al reescribir explícitamente las rutinas que antes encapsulaban las macros de `x86-bare-metal-examples/protected_mode.S`, se obtiene una perspectiva mucho más transparente del proceso subyacente.

La ejecución de este código se automatiza mediante el script `modo_protegido/run.sh`. Al ejecutar `./run.sh`, el sistema se encarga de compilar el ensamblador, generar el archivo binario correspondiente e inicializar QEMU para mostrar la salida. Además, este script facilita enormemente el entorno de depuración, ya que establece de forma automática los puntos de interrupción (*breakpoints*) necesarios en GDB.

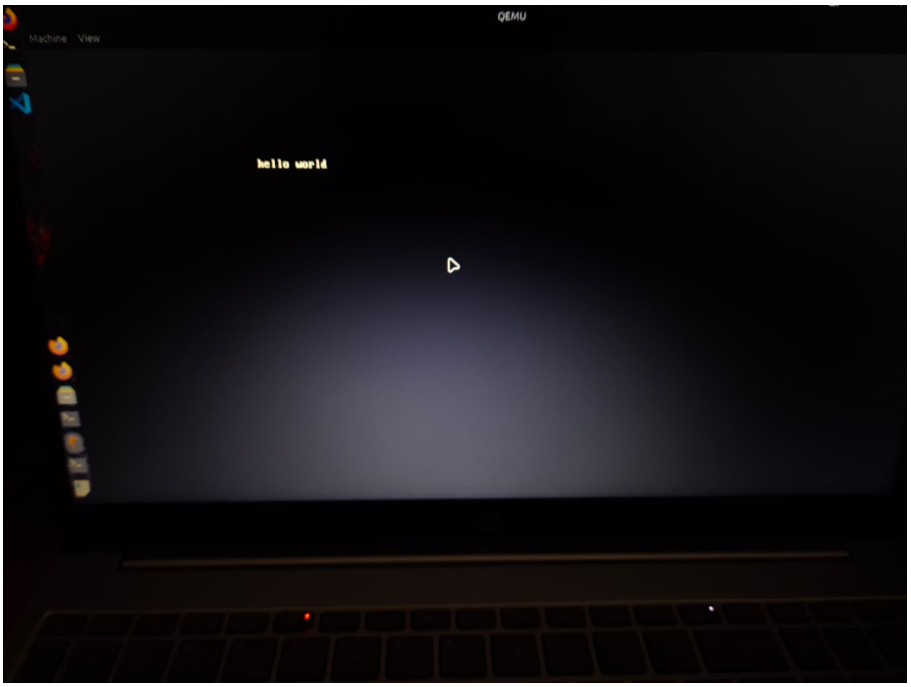
```
matias@matias:~/protected-mode-sdc/x86-bare-metal-examples$ ./run protected_mode
gcc -m32 -c -ggdb3 -o 'protected_mode.o' 'protected_mode.S'
ld -melf_i386  -nostdlib -o 'protected_mode.elf' -T 'linker.ld' 'protected_mode.o
'
objcopy -O binary 'protected_mode.elf' 'protected_mode.img'
qemu-system-i386 -drive 'file=protected_mode.img,format=raw' -serial mon:stdio -s
mp 2
```

Tras iniciar la sesión en GDB y confirmar la carga de los símbolos o etiquetas, la consola muestra el siguiente estado:

```
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from protected_mode.elf...
(gdb) target remote localhost:1234
Remote debugging using localhost:1234
0x0000fff0 in ?? ()
(gdb) break protected_mode
Punto de interrupción 1 at 0x7c7a: file protected_mode.S, line 6.
(gdb) continue
Continuando.

Breakpoint 1, 0x00007c7a in protected_mode () at protected_mode.S:6
6           PROTECTED_MODE
```

Tal como se observa, la ejecución se detiene en el *breakpoint* del modo protegido, exactamente antes de ingresar al ciclo. Si reanudamos la ejecución hasta el próximo punto de interrupción, el flujo del programa avanzará por la rutina de impresión, permitiendo visualizar el mensaje resultante directamente en la ventana del emulador QEMU.



Implementación de Múltiples Descriptores

El uso de descriptores de memoria independientes para las áreas de código y datos añade una capa vital de seguridad basada en el contexto. Al abandonar un modelo de memoria unificado y separar estos segmentos, logramos que:

- El segmento de código tenga permisos de ejecución, pero quede bloqueado para escritura.
- El segmento de datos admita operaciones de lectura y escritura, pero carezca totalmente de permisos de ejecución.

Esta arquitectura previene modificaciones indeseadas en la memoria, garantizando un aislamiento efectivo entre procesos y protegiendo la integridad de las instrucciones.

Restricción de Acceso: Segmento de Solo Lectura

El siguiente paso del análisis consiste en modificar los atributos de acceso del segmento de datos para convertirlo en un área de solo lectura. El propósito es observar la reacción de la arquitectura ante un intento de escritura no autorizado durante la ejecución del programa.

Para lograr esto, se deben remover los privilegios de escritura en el descriptor y volver a correr el entorno de la misma forma que en el caso anterior.

```
0x0000fff0 in ?? ()
(gdb) break *0x7c00
Punto de interrupción 1 at 0x7c00: file segmentation.S, line 5.
(gdb) c
Continuando.

Breakpoint 1, __start () at segmentation.S:5
5       BEGIN
(gdb) c
Continuando.
```

Figura 3: El programa se queda ahí

```
(gdb) info registers
eax            0x10                16
ecx            0x0                 0
edx            0x7e26              32294
ebx            0x7e00              32256
esp            0x7000              0x7000
ebp            0x7000              0x7000
esi            0x0                 0
edi            0x0                 0
eip            0x7f97              0x7f97 <message>
eflags         0x2                 [ IOPL=0 ]
cs             0x8                 8
ss             0x10                16
ds             0x10                16
es             0x10                16
fs             0x10                16
gs             0x10                16
fs_base        0x0                 0
gs_base        0x0                 0
k_gs_base      0x0                 0
cr0            0x11                [ ET PE ]
cr2            0x0                 0
cr3            0x0                 [ PDBR=0 PCID=0 ]
cr4            0x0                 [ ]
--Type <RET> for more, q to quit, c to continue without paging--x/10i $eip
cr8            0x0                 0
efer           0x0                 [ ]
```

Al inicializar `modo_protegido/run.sh` con esta nueva configuración restrictiva, se puede constatar mediante GDB que la ejecución queda atrapada en un bucle dentro del primer *breakpoint*. Esta interrupción ocurre porque el procesador detecta una violación de acceso al intentar escribir en un segmento donde no posee los permisos suficientes, generando una excepción a nivel de hardware.

Comportamiento de los Registros de Segmento

Al ingresar al modo protegido, los registros de segmento alteran su función: en lugar de almacenar valores directos para calcular direcciones, pasan a cargarse con *selectores de segmento*. Este cambio arquitectónico introduce una capa adicional de abstracción en la administración de la memoria, brindando al sistema un marco robusto que aumenta la flexibilidad, la seguridad y la compatibilidad.

La Tabla de Descriptores de Segmento (GDT)

La Tabla Global de Descriptores (GDT) o, en su defecto, la Tabla Local (LDT), es el mecanismo central que la CPU consulta en modo protegido para mapear, organizar y restringir el acceso a la memoria.

Esta estructura aloja los “descriptores”, los cuales definen con precisión las propiedades de cada segmento mediante los siguientes atributos:

Base: La dirección lineal exacta donde inicia el bloque de memoria.

Límite: El tamaño máximo o extensión permitida del segmento.

Permisos: Los privilegios concedidos, como lectura, escritura, ejecución y el nivel de privilegio requerido.

Tipo: La naturaleza de la información que aloja (segmento de código, de datos, de sistema, etc.).

En resumen, cada vez que se requiere un acceso a memoria, el procesador evalúa el selector, extrae el descriptor asociado de la tabla y valida si la operación cumple con las reglas establecidas. Esta mecánica es la piedra angular para segmentar la memoria e implementar políticas de protección y aislamiento rigurosas entre distintos espacios de trabajo.